

1 Introduction

- Group members: Pat Mutia
- Pandemaniac team name: Pathogen_Zero
- Division of labor:
 - *Overall*: I worked by myself and asked Ethan Dawn (TA) about the requirements, `sim.py`, and how to better approach infecting the graph to beat `TA_target`.
- Late Token(s): 1 late token were used for this report.

2 Overview

For week 1, I participated in all rounds. For the first day, I randomly selected nodes within the union of highest degree, closeness, and betweenness, which succeeded in beating a lot of teams and `TA_baseline` on the first day. For the next few days, I maintained the same strategy to see if adjusting the number of candidates I was considering would be successful in beating `TA_target`, but I gradually saw that it was beginning to lose and more importantly not beating `TA_target`. Therefore, I decided to focus on high degree for my next strategy and leverage the `sim.py` file for comparing the performance of seeds I selected. For Robin-Robin, I compared my the highest degree nodes and their generated seeds to find the best performing seeds for 4 minutes to allow for a minute for submitting the files. For Jungle, I decided to select the middle range of the highest degree nodes considering the rule of if multiple teams select the same node, then none of them receive that node. Completing this implementation on the last day of week 1, I was successful in beating `TA_target` and even tied with `TA_hard`. For the second week of competitions, I maintained the same strategies for Robin-Robin and Jungle, competing into the second round of the competition and overall winning the Jungle competition.

Due to the limited time and working by myself, I was not able to enhance my strategy more and implement more centrality measures into my second formulation of strategies. I could have considered implementing more different strategies based on graph type and the number of nodes.

3 Strategies

Centrality Measures

As my first approach at generating seeds for the provided networks, I decided to look into three different centrality measure that were available in the `networkx` package: degree, closeness, and betweenness. Degree centrality considers the total number of relationships connected to a particular node, so a node with high degree means that the epidemic could spread to more nodes during each iteration. Closeness centrality is the measure of the average shortest distance for each node to each other node or the inverse of the average shortest distance between the node and all other nodes in the network. Thus, a node with the highest closeness means its average path to other nodes is shorter, meaning that with the same steps, other nodes can be reached from nodes with higher closeness in fewer steps. Betweenness centrality is the measure of a node lying on the shortest paths of other pairs of nodes. This measures the importance of a node with respect to connecting other nodes in the path, identifying nodes that act as bridges between different parts of the graph.

Generating seeds with the highest degree, closeness, and betweenness, I noticed there was overlap in the selected nodes. Moreover, considering the rule where if more than one team pick the node, then none of the teams gets that node, I decided to do a combined strategy of the union of all of the centrality measures and randomly select from that union. For an initial attempt, this performed well for Robin-Robin competitions and beat TA.baseline on the first day. For the next few days, I kept the same strategy but played with ratio (e.g. 1.5) that controlled the number of candidate nodes I was considering within the graph, allowing for flexibility and randomness. Additionally, I noticed that calculating this would take a decent amount of time for larger graphs.

Robin Robin

Thus, I decided to focus on one centrality: high degree. Moreover, I wanted to leverage `sim.py` and create multiple seeds and try to find the best seed. For Robin-Robin, I took into account that the format of these rounds were head-to-head match ups, where each team competes against a single team at a given time. Then given that only one team can control a node at the start, the strategy I implemented focused on select the highest performing nodes from the highest degree nodes nodes with the most connection in order to quickly influence the neighboring nodes. In a Robin-Robin setting, it is important to control the well-connected nodes early on to provide a strong foundation for expansion, making it harder to the opponent group to overturn the spread. My implementation sorts nodes by degree, then uses `sim.py` to see and select the top-performing nodes as seeds. This ensures that the chosen nodes maximize initial influence. I chose this because countering my opponent's likely selection was also as important as picking high-performing nodes.

Jungle

For Jungle, I still chose to focus on high degree nodes. For Jungle, the competition was designed for a multi-team free-for-all competitions, so multiple teams are seeding nodes at the same time. Therefore in this format, selecting the highest degree node is at a higher risk of cancellation because multiple teams may target the same node in comparison to one other team in Robin-Robin. Therefore, I aimed for selecting nodes within the middle tier of degree range, so 3rd to 7th. Then added randomness in selecting the nodes

within this range, adding another measure to prevent opponents from predicting what I would select and thus countering it. So, the main focus was not participating in the direct competition over the most obvious high degree nodes to ensure spread and dominance in a free-for-all environment.

4 Testing

For comprehending the results of each day's competition, I looked at the visualization on the Pandemaniac website, which was helpful in looking at the structure of the graph that we were attempting to infect rather than the number of seeds infected through `sim.py`. However, I did have errors of loading the steps for some of the rounds, so this was not the main form of testing I used, but it did show me the importance of degree and clustering. For testing my algorithm, I would use `sim.py` on previous graphs and results from the TA's since TA Ethan told me that the strategies for the TA's do not change every day.

5 Conclusion

Overall, I was successful in beating `TA.baseline`, `TA.target`, and `TA.hard` (tie). For the official rounds, I competed until the second round. Moreover, I won the official Jungle competition where I competed against the other teams. I achieved this through focusing on the centrality measure of highest degree and leveraging the `sim.py` for iterating through various seeds for Robin-Robin. For Jungle, I avoided attempting to infect the first few max degree nodes to avoid failing to infect the node by rules of the Pandemaniac competition.

6 Suggestions

My main obstacle was forming a group for this mini-project. I had reached out to people way ahead of time for the competition through emails and messages. I also asked the TAs about who had not formed a group yet in the class on Piazza and publicized that post for people to reach out to me. Overall, it resulted in a mix up over teams that forgot to report their team members before the deadline. This is a more a reflection of the students of the class but this could have been mitigated through openly reporting who were on teams through something like a Google Sheets spreadsheet. I found that I had been asking multiple people within the same group and not receiving messages back, spending time that I could have been working on setting up any utils functions for the competition. TA Ethan was extremely helpful and transparent about sorting out the group situation and how difficult it would be to work alone. He answered my questions quickly and helped with some errors I had over the Pandemaniac website.